

Chapter 03

Algorithm Development and Control Statements: Part 1

Learning Objectives

- Learn basic problem-solving techniques.
- Develop algorithms through the process of top-down, stepwise refinement.
- Use the if and if...else selection statements to choose between alternative actions.
- Use the while iteration statement to execute statements in a program repeatedly.
- Use counter-controlled iteration and sentinel-controlled iteration.
- Use nested control statements.
- Use the compound assignment operators and the increment and decrement operators.
- Learn why fundamental data types are not portable.
- Continue our Objects Natural approach with a case study on creating and manipulating integers as large as you want them to be.

Introduction

- Before writing a program to solve a problem, you should have a thorough understanding of it and a carefully planned approach to solving it.
- When writing a program, you also should understand the available building blocks and employ proven program-construction techniques.
- In this chapter and the next, we present the theory and principles of structured programming. The concepts presented here will be crucial in building classes and manipulating objects.

Introduction.

- We discuss the `if` statement in additional detail and introduce the `if...else` and `while` statements—all of these building blocks allow you to specify the logic required for functions to perform their tasks.
- We also introduce the compound assignment operators and the increment and decrement operators.
- We discuss why the fundamental types are not portable.
- We continue our Objects Natural approach with a case study on creating and manipulating super-sized integers that can represent values beyond the ranges of integers supported by computer hardware.

Algorithms

- The solution to any computing problem involves executing a series of actions in a specific order.
- A **procedure** for solving a problem in terms of
 - the **actions** to be executed, and
 - the **order** in which these actions are to be executedis called algorithm.
- Correctly specifying the order in which the actions are to be executed is important.

Algorithms (Cont.)

- Consider the “rise-and-shine algorithm” followed by one junior executive for getting out of bed and going to work:
 - 1) Get out of bed
 - 2) Take off pajamas
 - 3) Take a shower
 - 4) Get dressed
 - 5) Eat breakfast
 - 6) Carpool to work
- This routine gets the executive to work well prepared to make critical decisions.

Algorithms.

- Suppose that the same steps are performed in a slightly different order
 - 1) Get out of bed
 - 2) Take off pajamas
 - 3) Get dressed
 - 4) Take a shower
 - 5) Eat breakfast
 - 6) Carpool to work
- In this case, our junior executive shows up for work soaking wet.
- Specifying the order in which statements are to be executed in a computer program is called **program control**.

Pseudocode

- **Pseudocode** is an informal language that helps you develop algorithms without worrying about the strict details of C++ language syntax.
 - The pseudocode we present is particularly useful for developing algorithms that will be converted to structured portions of C++ programs.
- Pseudocode is similar to everyday English (or your preferred language)
 - it's convenient and user-friendly, but it's not an actual computer programming language.
 - The style of pseudocode we present consists purely of characters, so you can type pseudocode conveniently using any text-editor program.
- Pseudocode does not execute on computers.
 - Rather, it helps you “think out” a program before attempting to write it in a programming language like C++.

Pseudocode.

- A carefully prepared pseudocode program can easily be converted to a corresponding C++ program.
- Pseudocode typically describes only statements representing the **actions** that occur after you convert a program from pseudocode to C++ and run the program
 - Such actions might include input, output, assignments or calculations. In our pseudocode
 - We typically do **NOT** include **variable declarations**, but some programmers choose to list variables and mention their purposes.

Control Structures

- Sequential execution
 - Normally, statements in a program are **executed one after the other** in the order in which they're written.
- Transfer of control
 - Various C++ statements we'll soon discuss enable you to specify that the **next statement to be executed** may be **other than the next one in sequence**.
- During the 1960s, it became clear that the indiscriminate use of transfers of control was the **root of** a great deal of **difficulty** experienced by software development groups.

Control Structures (Cont.)

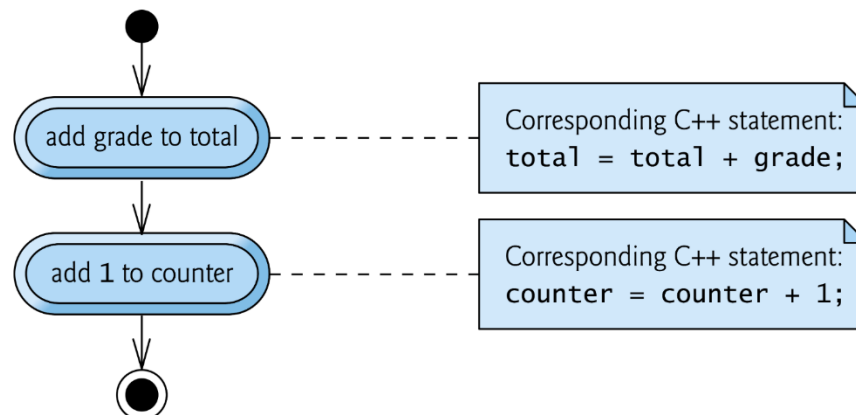
- The finger of blame was pointed at the `goto statement` that allows you to specify a transfer of control to one of many possible destinations in a program.
- Structured programming became almost synonymous with “`goto elimination.`”
- Research had demonstrated that programs `could be written without any goto` statements.
- The challenge of the era was for programmers to shift their styles to “`goto-less programming.`”

Control Structures.

- The results were impressive, as software development groups reported **reduced development times**, more frequent **on-time delivery of systems** and more frequent **within-budget completion** of software projects.
- Programs produced with structured techniques were **clearer, easier to debug** and **modify** and more likely to be bug free in the first place.
- Research had demonstrated that all programs could be written in terms of only three **control structures**, namely the **sequence structure**, the **selection structure** and the **iteration structure**.

Sequence Structure

- The **sequence structure** is built into C++.
 - Unless directed otherwise, statements execute one after the other in the order they appear in the program—that is, in sequence.
 - The following **unified modeling language (UML)** activity diagram illustrates a typical sequence structure in which two calculations are performed in order:



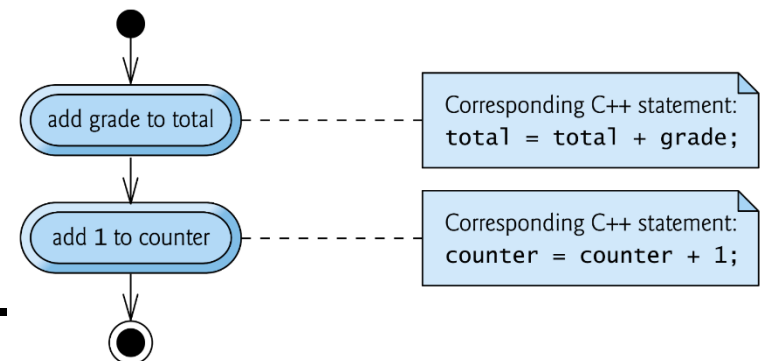
Sequence Structure (Cont.)

- C++ lets you have as many actions as you want in a sequence structure.
- An activity diagram models the **workflow** (also called the **activity**) of a portion of a software system.
 - Such workflows may include a portion of an algorithm, like the sequence structure in the preceding diagram.
 - Activity diagrams are composed of symbols, such as
 - ◆ action-state symbols (rectangles with their left and right sides replaced with outward arcs),
 - ◆ diamonds and
 - ◆ small circles.
 - These symbols are connected by transition arrows, representing the activity's flow—that is, the order in which the actions should occur.

Like pseudocode, activity diagrams help you develop and represent algorithms. Activity diagrams clearly show how control structures operate.

Sequence Structure (Cont.)

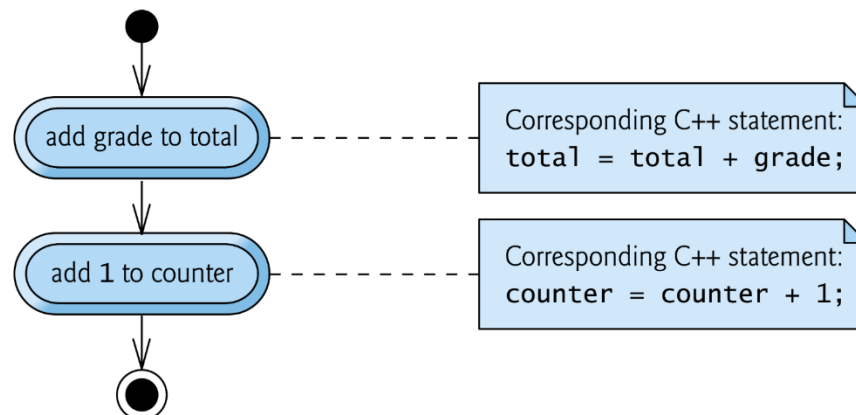
- The sequence-structure activity diagram
 - contains two action states, each containing an action expression.
 - The arrows in the activity diagram represent transitions, which indicate the order in which the actions represented by the action states occur.
- The program that implements the activities illustrated in this activity diagram first adds grade to total, then adds 1 to counter.



Sequence Structure (Cont.)

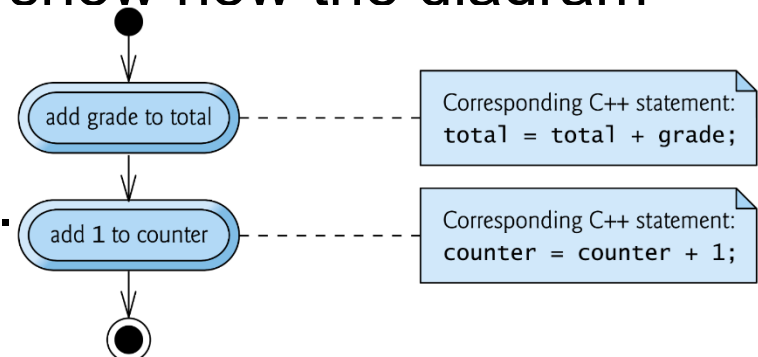
- The Solid Circle

- at the top of the activity diagram represents the initial state—the beginning of the workflow before the program performs the modeled actions.
- surrounded by a hollow circle at the bottom of the diagram represents the final state—the end of the workflow after the program performs its actions.



Sequence Structure.

- The preceding sequence-structure activity diagram also includes rectangles with the upper-right corners folded over.
 - These are UML notes (like C++ comments)—explanatory remarks describing the purpose of symbols in the diagram.
 - A dotted line connects a note with what it describes.
 - This diagram's UML notes show how the diagram relates to the C++ code.
 - Activity diagrams usually do not show the C++ code.



Selection Statements

- Three selection statements in C++:
 - The `if` selection statement
 - ◆ Either selects (performs) an action if a condition is true or skips the action if the condition is false
 - ◆ **Single-selection** statement: it selects or ignores a single action.
 - The `if...else` selection statement
 - ◆ Performs an action if a condition is true and performs a different action if the condition is false.
 - ◆ **Double-selection** statement: it selects between two different actions.
 - The `switch` selection statement
 - ◆ Performs one of many different actions depending on the value of an expression.
 - ◆ **Multiple-selection** statement: it selects among many different actions

Iteration Statements

- **Iteration statements** — also called repetition statements or looping statements — are used for performing statements repeatedly while a **loop-continuation condition** remains true.
- Four iteration statements in C++:
 - `while`, `do...while`, `for`, and `range-based for`.
 - The `while` and `for` statements perform their action (or group of actions) zero or more times.
 - ◆ If the loop-continuation condition is initially false, the action (or group of actions) does not execute.
 - The `do...while` statement performs its action (or group of actions) one or more times.

C++ keywords

alignas	alignof	and	and_eq	asm
auto	bitand	bitor	bool	break
case	catch	char	char16_t	char32_t
class	compl	const	const_cast	constexpr
continue	decltype	default	delete	do
double	dynamic_cast	else	enum	explicit
export	extern	false	final	float
for	friend	goto	if	import
inline	int	long	module	mutable
namespace	new	noexcept	not	not_eq
nullptr	operator	or	or_eq	override
private	protected	public	register	reinterpret_cast
return	short	signed	sizeof	static
static_assert	static_cast	struct	switch	template
this	thread_local	throw	true	try
typedef	typeid	typename	union	unsigned
using	void	volatile	virtual	wchar_t
while	xor	xor_eq		

Keywords new in C++20

char8_t	concept	constexpr	constinit	co_await
co_return	co_yield	requires		

Summary of Control Statements

- C++ has only three kinds of control structures, which from this point forward, we refer to as control statements:
 - sequence,
 - selection (`if`, `if...else` and `switch`) and
 - iteration (`while`, `do...while`, `for`, and range-based `for`).

Summary of Control Statements.

- You form every program by combining these statements as appropriate for the algorithm you're implementing.
 - We can model each control statement as an activity diagram.
 - Each diagram contains initial and final states representing a control statement's entry and exit points.
 - **Single-entry/single-exit control statements** make it easy to build readable programs—we simply connect the exit point of one to the entry point of the next using **control-statement stacking**.
 - There's only one other way in which you may connect control statements—**control-statement nesting**, in which one control statement appears inside another.
- Thus, algorithms in C++ programs are constructed from only three kinds of control statements combined in only two ways. This is the essence of simplicity.

`if` Single-Selection Statement

- Programs use selection statements to choose among alternative courses of action.
 - For example, suppose that the passing grade on an exam is 60. The following pseudocode represents an `if` statement that tests the condition “student’s grade is greater than or equal to 60”:

```
if student's grade is greater than or equal to 60
    print "Passed"
```

- If this **condition** is true, the statement prints “Passed”—otherwise, the print statement is ignored.
- **Indenting** the second line of this selection statement is optional but recommended for clarity—it emphasizes that the print statement is in the if statement’s body.

if Single-Selection Statement (Cont.)

- The preceding pseudocode

```
if student's grade is greater than or equal to 60  
    print "Passed"
```

may be written in C++ as

```
if (studentGrade >= 60) {  
    cout << "Passed";  
}
```

- The C++ code corresponds closely to the pseudocode.

`if` Single-Selection Statement (Cont.)

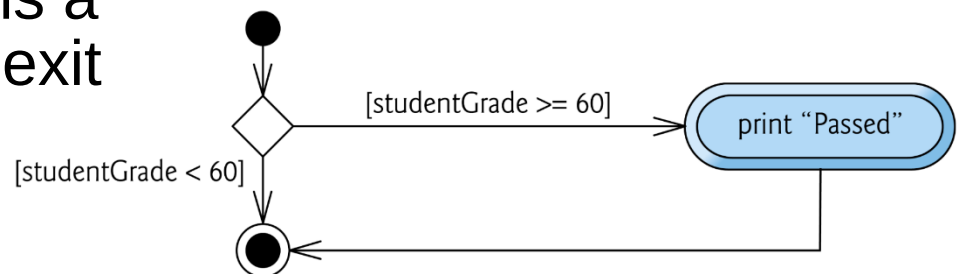
- `bool` Data Type

- Conditions can be created using the relational or equality operators.
- Actually, any expression that evaluates to zero or nonzero can be used as a condition.
 - ◆ Zero is treated as false, and
 - ◆ nonzero is treated as true.
- C++ also provides the `bool` data type for Boolean variables that can hold only the values `true` and `false`—each is a C++ keyword.
- The compiler can implicitly convert `true` to 1 and `false` to 0.

`if` Single-Selection Statement.

- UML Activity Diagram for an `if` Statement

- The most important symbol in an activity diagram—the diamond, or decision symbol, which indicates that a decision is to be made.
- The workflow continues along a path determined by the symbol's associated **guard conditions**, which can be true or false.
 - ◆ Each transition arrow emerging from a UML decision symbol has a guard condition (specified in square brackets next to the arrow).
- The `if` statement is a single-entry/single-exit control statement.



`if...else` Double-Selection Statement

- The `if...else` double-selection statement allows you to specify an action to perform when the condition is true and another action when the condition is false.
 - For example, the following pseudocode represents an `if...else` statement that prints “Passed” if the student’s grade is greater than or equal to 60 but prints “Failed” if it’s less than 60:

```
if student's grade is greater than or equal to 60
    print "Passed"
else
    print "Failed"
```

if...else Double-Selection Statement (Cont.)

- The preceding pseudocode statement can be written in C++ as

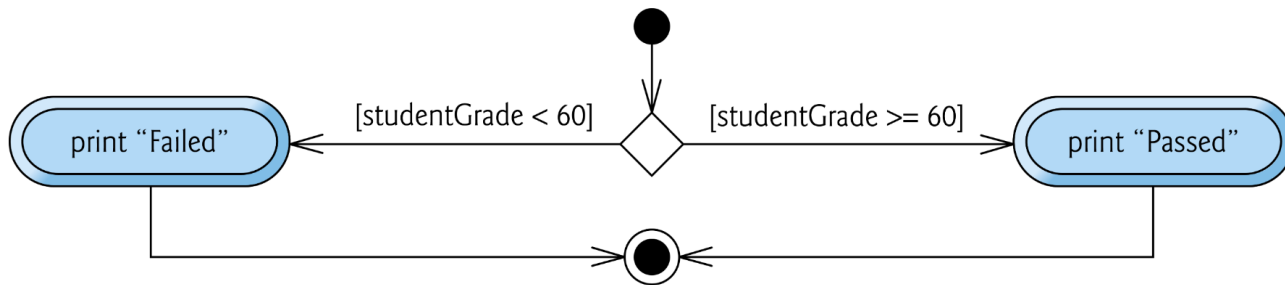
```
if (studentGrade >= 60) {  
    cout << "Passed";  
}  
else {  
    cout << "Failed";  
}
```

- The `if` and `else` bodies are both indented equally.
 - Whatever indentation convention you choose should be applied consistently throughout your programs.
 - If there are several indentation levels, each should be indented the same additional amount of space.

if...else Double-Selection Statement.

- UML Activity Diagram for an if...else Statement

- The following diagram illustrates the flow of control in the preceding statement:



Nested `if...else` Statement

- A program can test multiple cases by placing `if...else` statements inside other `if...else` statements to create nested `if...else` statements.
- For example, the following pseudocode represents a nested `if...else` statement that prints 'A' for exam grades greater than or equal to 90, 'B' for grades 80 to 89, 'C' for grades 70 to 79, 'D' for grades 60 to 69 and 'F' for all other grades.

Nested `if...else` Statement (Cont.)

```
if student's grade is greater than or equal to 90
    print "A"
else
    if student's grade is greater than or equal to 80
        print "B"
    else
        if student's grade is greater than or equal to 70
            print "C"
        else
            if student's grade is greater than or equal to 60
                print "D"
            else
                print "F"
```

Nested if...else Statement (Cont.)

- This pseudocode may be written in C++ as:

```
if ( studentGrade >= 90 )  
    cout << "A";  
else  
    if (studentGrade >= 80 )  
        cout << "B";  
    else  
        if (studentGrade >= 70 )  
            cout << "C";  
        else  
            if (studentGrade >= 60 )  
                cout << "D";  
            else  
                cout << "F";
```

Nested if...else Statement (Cont.)

- The preceding statement also can be written with fewer braces, less spacing and indentation:

```
if ( studentGrade >= 90 )  
    cout << "A";  
else if (studentGrade >= 80 )  
    cout << "B";  
else if (studentGrade >= 70 )  
    cout << "C";  
else if (studentGrade >= 60 )  
    cout << "D";  
else  
    cout << "F";
```

Nested `if...else` Statement.

- Above form avoids deep indentation of the code to the right, which sometimes can force lines to wrap.
- Enclose control-statement bodies in braces (`{` and `}`) can avoid a logic error called the “`dangling-else`” problem.

Blocks

- Control statement expects only **one** statement in its body.
- To include several statements, enclose the statements in **braces**.
 - It's good practice always to use braces.
- Statements in a pair of braces form a **block**.
 - A block can be placed anywhere in a function where a single statement can be placed.

Blocks (Cont.)

- The following example includes a block of multiple statements in an `if...else` statement's **else part**:

```
if (studentGrade >= 60) {  
    cout << "Passed";  
}  
else {  
    cout << "Failed\n";  
    cout << "You must retake this course.";  
}
```

Blocks.

- Without the braces surrounding the two statements in the `else` clause, the statement `cout << "You must retake this course.";` would be outside the body of the `else` part of the `if...else` statement and would execute regardless of whether the condition is met—a logic error.

Syntax and Logic Errors

- The compiler catches **syntax errors**, such as when one brace in a block is left out of the program.
- A **logic error**, such as an incorrect calculation, has its effect at execution time.
 - A **fatal logic error** causes a program to fail and terminate prematurely.
 - A **nonfatal logic error** allows a program to continue executing but causes it to produce incorrect results.

Empty Statement

- Just as a block can be placed anywhere a single statement can be placed, it's also possible to have an **empty statement**, which is simply a semicolon (;) where a statement typically would be.
 - An empty statement has **no effect**.
- Placing a semicolon after the parenthesized condition in an `if` or `if...else` statement leads to
 - a logic error in single-selection `if` statements and
 - a syntax error in double-selection `if...else` statements (when the `if`-part contains a body statement).

Conditional Operator

- C++ provides the **conditional operator** (**? :**) which is closely related to the **if...else** statement.
- The conditional operator is **C's only ternary operator**—it takes **three operands**. These together with the conditional operator form a conditional expression.
 - The **first** operand is a **condition**.
 - The **second operand** is **the value** for the entire conditional expression if the condition is **true** and the **third operand** is **the value** for the entire conditional expression if the condition is **false**.

Conditional Operator.

- For example, the statement

```
cout << (grade >= 60 ? "Passed" : "Failed");
```

contains as its second argument a conditional expression that evaluates to the string "Passed" if the condition `grade >= 60` is true and to the string "Failed" if the condition is false.

- The operand to the left of the `?` is a condition. The second operand (between the `?` and `:`) is the conditional expression's value if the condition is true. The operand to the right of the `:` is the conditional expression's value if the condition is false.

while Iteration Statement

- An iteration statement allows you to specify that a program should repeat an action while some condition remains true.
- The pseudocode statement

*while there are more items on my shopping list
purchase next item and cross it off my list*

describes the iteration during a shopping trip.

- The condition “there are more items on my shopping list” may be true or false.
 - ◆ If it’s true, the action “purchase next item and cross it off my list” is performed.
 - ◆ This action will be performed repeatedly while the condition remains true. The statement(s) contained in the **while iteration statement** constitute its body, which may be a single statement or a block.
 - ◆ Eventually, the condition will become false. At this point, the iteration terminates, and the first statement after the iteration statement executes.

while Iteration Statement (Cont.)

- As a `while` iteration statement example, consider the following program segment that finds the first power of 3 larger than 100:

```
int product{3};  
    while (product <= 100) {  
        product = 3 * product;  
    }
```

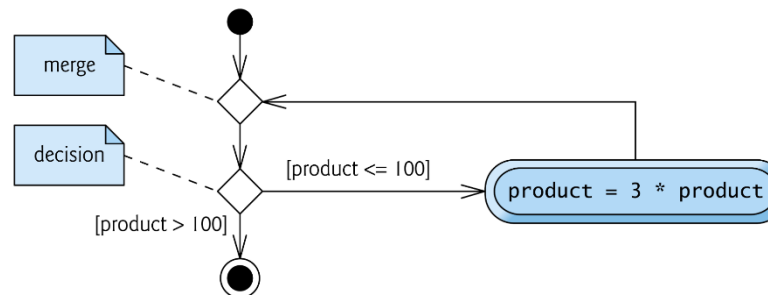
- Each iteration of the `while` statement multiplies `product` by 3, so `product` takes on the values 9, 27, 81 and 243 successively.
- When `product` becomes 243, `product <= 100` becomes false.
- This terminates the iteration, so the final value of `product` is 243.
- At this point, program execution continues with the next statement after the `while` statement.
- Not providing in a `while` statement's body an action that eventually causes the condition to become false results in a logic error called an **infinite loop** (the loop never terminates).

while Iteration Statement (Cont.)

- UML Activity Diagram for a `while` Statement

- The following `while` statement UML activity diagram introduces the **merge symbol**

- ◆ The UML represents both the merge symbol and the decision symbol as diamonds.
 - ◆ The merge symbol joins two flows of activity into one.
 - In this diagram, the merge symbol joins the transitions from the initial state and the action state, so they both flow into the decision that determines whether the loop should begin (or continue) executing.



while Iteration Statement.

- UML Activity Diagram for a `while` Statement

- You can distinguish the decision and merge symbols by the number of incoming and outgoing transition arrows.

- ◆ A decision symbol

- has one transition arrow pointing to the diamond and two or more pointing out from it to indicate possible transitions from that decision.
- Also, each arrow pointing out of a decision symbol has a guard condition.

- ◆ A merge symbol

- has two or more transition arrows pointing to it, and only one pointing from it to indicate multiple activity flows merging to continue the activity.
- None of the transition arrows associated with a merge symbol has a guard condition.

Formulating Algorithms: Counter-Controlled Iteration

- To illustrate how algorithms are developed, we solve two variations of a problem that averages student grades. Consider the following problem statement:

A class of ten students took a quiz. The quiz grades (integers in the range 0–100) are available to you. Determine the class average on the quiz.

- The class average is equal to the sum of the grades divided by the number of students. The program must input each grade, total all the grades entered, perform the averaging calculation and print the result.

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Pseudocode Algorithm with Counter-Controlled Iteration

- Let's use pseudocode to list the actions to execute and specify the order in which they should execute.
- We use **counter-controlled iteration** to input the grades one at a time.
 - ◆ This technique uses a variable called a **counter** (or **control variable**) to control the number of times a set of statements will execute.
 - ◆ Counter-controlled iteration is often called **definite iteration** because the number of iterations is known before the loop begins executing.
 - ◆ In this example, iteration terminates when the counter exceeds 10.
 - ◆ This section presents the following fully developed pseudocode algorithm and a corresponding C++ program that implements the algorithm.

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Pseudocode Algorithm with Counter-Controlled Iteration

set total to zero

set grade counter to one

while grade counter is less than or equal to ten

prompt the user to enter the next grade

input the next grade

add the grade into the total

add one to the grade counter

set the class average to the total divided by ten

print the class average

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Note the references in the preceding algorithm to a total and a counter.
 - A total is a variable used to accumulate the sum of several values.
 - ◆ Variables used to store totals normally are **initialized to zero** before being used in a program.
 - A counter is a variable used to count—the grade counter indicates which of the 10 grades is about to be entered by the user.
 - In pseudocode, we do **NOT** use braces around the statements that form the pseudocode while's body, but you could.
- Experience has shown that the most challenging part of solving a problem on a computer is **developing the algorithm** for the solution.
 - Once a correct algorithm has been specified, producing a working C++ program from it is usually straightforward.

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

```
1 // Solving the class-average problem using counter-controlled iteration.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // initialization phase
7     int total{0}; // initialize sum of grades entered by the user
8     int gradeCounter{1}; // initialize grade # to be entered next
9
10    // processing phase uses counter-controlled iteration
11    while (gradeCounter <= 10) { // loop 10 times
12        cout << "Enter grade: "; // prompt
13        int grade;
14        cin >> grade; // input next grade
15        total = total + grade; // add grade to total
16        gradeCounter = gradeCounter + 1; // increment counter by 1
17    }
18
19    // termination phase
20    int average{total / 10}; // int division yields int result
21
22    // display total and average of grades
23    cout << "\nTotal of all 10 grades is " << total;
24    cout << "\nClass average is " << average << "\n";
25 }
```

Enter grade: **67**
Enter grade: **78**
Enter grade: **89**
Enter grade: **67**
Enter grade: **87**
Enter grade: **98**
Enter grade: **93**
Enter grade: **85**
Enter grade: **82**
Enter grade: **100**

Total of all 10 grades is 846
Class average is 84

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

• Local Variables

- Lines 7, 8, 13 and 20 declare `int` variables.
 - ◆ Variable `grade` stores the user input.
 - ◆ A variable declared in a block (such as a function's body) is a **local variable** that can be used only from the line of its declaration to the closing right brace of the block.
 - A local variable's declaration must appear before the variable is used.
 - Variable `grade`—declared at line 13 the `while` loop's body—can be used only in that block.

```
7 | int total{0}; // initialize sum of grades entered by the user
8 | int gradeCounter{1}; // initialize grade # to be entered next
13 | int grade;
20 | int average{total / 10}; // int division yields int result
```

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Initializing Variables

- Lines 7–8 declare and initialize `total` to 0 and `gradeCounter` to 1.
- These initializations occur before the variables are used in calculations.
- Initialize each total and counter, either in its declaration or in an assignment statement.
- Totals are typically initialized to 0 and counters are typically initialized to 0 or 1, depending on their use.

```
7 | int total{0}; // initialize sum of grades entered by the user  
8 | int gradeCounter{1}; // initialize grade # to be entered next
```

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Reading 10 Grades from the User
 - The `while` statement (lines 11–17) continues iterating as long as `grade`'s value is less than or equal to 10.
 - Line 12 displays the prompt.
 - Line 14 inputs the grade entered by the user and assigns it to variable `grade`.

```
11 while (gradeCounter <= 10) { // loop 10 times
12     cout << "Enter grade: "; // prompt
13     int grade;
14     cin >> grade; // input next grade
15     total = total + grade; // add grade to total
16     gradeCounter = gradeCounter + 1; // increment counter by 1
17 }
```

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Reading 10 Grades from the User

- Then line 15 adds the new `grade` entered by the user to the `total` and assigns the result to `total`, replacing its previous value.
- Line 16 adds 1 to `gradeCounter` to indicate that the program has processed a grade and is ready to input the next grade from the user.
 - ◆ Incrementing `gradeCounter` eventually causes it to exceed 10, which terminates the loop.

```
11  while (gradeCounter <= 10) { // loop 10 times
12      cout << "Enter grade: "; // prompt
13      int grade;
14      cin >> grade; // input next grade
15      total = total + grade; // add grade to total
16      gradeCounter = gradeCounter + 1; // increment counter by 1
17  }
```

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Calculating and Displaying the Class Average
 - When the loop terminates, line 20 performs the averaging calculation in the `average` variable's initializer.
 - Line 23 displays the text followed by variable `total`'s value.
 - Then, line 24 displays the text followed by `average`'s value.

```
19 // termination phase
20 int average{total / 10}; // int division yields int result
21
22 // display total and average of grades
23 cout << "\nTotal of all 10 grades is " << total;
24 cout << "\nClass average is " << average << "\n";
```

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Integer Division and Truncation

- This example's average calculation produces an int result.
- The program's sample execution shows that the sum of the grades is 846—when divided by 10, this should yield 84.6.
- Numbers like 84.6 containing decimal points are **floating-point numbers**.
- However, in the class-average program, `total / 10` produces the integer 84 because `total` and `10` are both integers.
- Dividing two integers results in **integer division**—any fractional part of the calculation is **truncated**.
 - ◆ For example, $7/4$ yields 1.75 in conventional arithmetic but truncates to 1 in integer arithmetic rather than rounding to 2.
 - ◆ Assuming that integer division rounds (rather than truncates) can lead to incorrect results.

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Arithmetic Overflow

- The following statement

```
total = total + grade;
```

adds `grade` entered by the user to the `total`.

- Even this simple statement has a potential problem—adding the integers could result in a value too large to store in an `int` variable.
- This is known as **arithmetic overflow** and causes undefined behavior, which can lead to unintended results and security problems.

Formulating Algorithms: Counter-Controlled Iteration (Cont.)

- Arithmetic Overflow

- The following expressions return your system's maximum and minimum `int` values:

```
std::numeric_limits<int>::max()  
std::numeric_limits<int>::min()
```

- Each uses class `numeric_limits` from the C++ standard library header `<limits>`.
- You may use similar expressions for any integral or floating-point type by replacing `int` in these expressions.

Formulating Algorithms: Counter-Controlled Iteration.

- Input Validation

- When a program receives input from the user, various problems might occur, e.g.,

`cin >> grade;`

assumes that the user will enter an integer grade from 0 to 100 but the user could enter:

- ◆ an integer less than 0, an integer greater than 100,
 - ◆ an integer outside the range of values that can be stored in an `int` variable,
 - ◆ a number containing a decimal point or
 - ◆ a value containing letters or special symbols that do not even represent a number.
 - Industrial-strength programs must test for all possible erroneous cases to ensure valid inputs.
 - ◆ A program that inputs grades should validate them using **range checking** to ensure they are values from 0 to 100.
 - ◆ If a program requires inputs from a specific set of values (such as, non-sequential product codes), you should ensure that each input matches a value in the set.

Formulating Algorithms: Sentinel-Controlled Iteration

- Let's generalize the class-average problem. Consider the following problem:

Develop a class-averaging program that processes grades for an arbitrary number of students each time it is run.

For this problem, we do not know how many grades the user will enter during the program's execution.

- The program must process an arbitrary number of grades.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- One way to solve this problem is to use a **sentinel value** (also called a signal value or a flag value) to indicate the end of data entry.
 - The user enters grades until all legitimate grades have been entered.
 - The user then enters the sentinel value to indicate that no more grades will be entered.
- You must choose a sentinel value that cannot be confused with an acceptable input value.
 - Grades on a quiz are non-negative integers, so -1 is an acceptable sentinel value for this problem.
 - Thus, a run of the class-averaging program might process a stream of inputs such as 95, 96, 75, 74, 89 and -1.
 - The program would then compute and print the class average for the grades, excluding -1, because it's the sentinel value and should not enter into the averaging calculation.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- The user could enter -1 before entering grades, which means the number of grades will be zero.
- We must test for this case before calculating the class average.
- According to the C++ standard, the result of division by zero in floating-point arithmetic is undefined.
 - When performing division (/) or remainder (%) calculations in which the right operand could be zero, test for this and handle it (e.g., display an error message) rather than allowing the calculation to proceed.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Top-Down, Stepwise Refinement

- We approach the class-average program with a technique called top-down, stepwise refinement, a technique that is essential to the development of well-structured programs.
- We begin with a pseudocode representation of the top:
 - ◆ Determine the class average for the quiz
- The top is a single statement that conveys the program's overall function.
- As such, the top is, in effect, a complete representation of a program.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Top-Down, Stepwise Refinement

- Unfortunately, the **top rarely conveys** a sufficient amount of **detail** for writing the C program.
- So we now begin the **refinement process**.
- We **divide the top into a series of smaller tasks** and **list** these **in** the **order** they need to be performed.
- This results in the following first refinement.
 - ◆ **Initialize variables**
Input, sum, and count the quiz grades
Calculate and print the class average
- Here, only the sequence structure has been used—the steps listed are to be executed in order, one after the other.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Second Refinement

- To proceed to the next level of refinement, i.e., the second refinement, we commit to specific variables.
- We need a running total of the numbers, a count of how many numbers have been processed, a variable to receive the value of each grade as it's input and a variable to hold the calculated average.
- The pseudocode statement
 - ◆ Initialize variables
- may be refined as follows:
 - ◆ Initialize total to zero
 - Initialize counter to zero

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Second Refinement

- Only the total and counter need to be initialized; the variables average and grade (for the calculated average and the user input, respectively) need not be initialized because their values will be written over by the process of destructive read-in discussed in Chapter 2.
- The pseudocode statement
Input, sum, and count the quiz grades
requires an iteration structure that successively
inputs each grade.
- Because we do not know in advance how many grades are to be processed, we'll use sentinel-controlled iteration.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Second Refinement

- The user will enter legitimate grades in one at a time.
- After the last legitimate grade is typed, the user will type the sentinel value.
- The program will test for this value after each grade is input and will terminate the loop when the sentinel is entered.
- The refinement of the preceding pseudocode statement is then
 - ◆ Input the first grade
 - While the user has not as yet entered the sentinel
 - Add this grade into the running total
 - Add one to the grade counter
 - Input the next grade (possibly the sentinel)

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Second Refinement

- Notice that in **pseudocode**, we do **not use braces** around the set of statements that form the body of the while statement.
- We simply **indent** all these statements under the while to show that they all belong to the while.
- Again, pseudocode is an informal program development aid.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Second Refinement

- The pseudocode statement

- ◆ Calculate and print the class average

- ◆ may be refined as follows:

- ◆ If the counter is not equal to zero

- Set the average to the total divided by the counter

- Print the average

- else

- Print “No grades were entered”

- We’re careful here to test for the possibility of **division by zero**—a logic error that, if undetected, would cause the program to fail or produce invalid output.

- ◆ According to the C++ standard, the result of division by zero in floating-point arithmetic is **undefined behavior**.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Here is the complete second refinement:

```
initialize total to zero
initialize counter to zero

prompt the user to enter the first grade
input the first grade (possibly the sentinel)

while the user has not yet entered the sentinel
    add this grade into the running total
    add one to the grade counter
    prompt the user to enter the next grade
    input the next grade (possibly the sentinel)

if the counter is not equal to zero
    set the average to the total divided by the counter
    print the average
else
    print "No grades were entered"
```

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- We use **blank lines** to make the pseudocode more readable and **indentation** to emphasize the actions that should be placed in the control statements' bodies.
 - The blank lines separate the algorithms into their phases and set off control statements;
 - the indentation emphasizes the bodies of the control statements.
- Our pseudocode algorithm solves the more general class-average problem.
 - This algorithm required two refinements—sometimes, more are needed.
 - You terminate the top-down, stepwise refinement process when you feel your pseudocode has sufficient detail for you to convert it to C++.
- Some programmers do not use program development tools like pseudocode.
 - They feel that their ultimate goal is to solve the problem on a computer and that writing pseudocode merely delays the final product.
 - This may work for simple and familiar problems but can lead to significant errors and delays in large, complex projects.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Implementing Sentinel-Controlled Iteration
 - Next program implements [sentinel-controlled iteration](#).
 - Although each grade entered by the user is an integer, the average calculation will likely produce a floating-point number, which an `int` cannot represent.
 - C++ provides types `float`, `double` and `long double` to store [floating-point numbers](#):
 - ◆ A `double` variable typically stores numbers with larger magnitude and finer detail than a `float`—that is, more digits to the right of the decimal point, also known as the number's [precision](#).
 - ◆ Similarly, a `long double` variable typically stores values with larger magnitude and more precision than `double`.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Implementing Sentinel-Controlled Iteration

- Initialization Phase

```
1 // Solving the class-average problem using sentinel-controlled iteration.
2 #include <iostream>
3 #include <iomanip> // parameterized stream manipulators
4 using namespace std;
5
6 int main() {
7     // initialization phase
8     double total{0.0}; // initialize sum of grades
9     int gradeCounter{0}; // initialize # of grades entered so far
```

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Implementing Sentinel-Controlled Iteration
 - Processing Phase

```
11 // processing phase
12 // prompt for input and read grade from user
13 cout << "Enter grade or -1 to quit: ";
14 int grade;
15 cin >> grade;
16
17 // loop until sentinel value is read from user
18 while (grade != -1) {
19     total = total + grade; // add grade to total
20     gradeCounter = gradeCounter + 1; // increment counter
21
22     // prompt for input and read next grade from user
23     cout << "Enter grade or -1 to quit: ";
24     cin >> grade;
25 }
```

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Implementing Sentinel-Controlled Iteration

- Termination Phase

```
27 // termination phase
28 // if user entered at least one grade
29 if (gradeCounter != 0) { // avoid division by zero
30     // calculate average of grades
31     double average(total / gradeCounter);
32
33     // display total and average (with two digits of precision)
34     cout << "\nTotal of the " << gradeCounter
35         << " grades entered is " << total;
36     cout << setprecision(2) << fixed;
37     cout << "\nClass average is " << average << "\n";
38 }
39 else { // no grades were entered, so output appropriate message
40     cout << "No grades were entered\n";
41 }
42 }
```

Enter grade or -1 to quit: -1
No grades were entered

Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1

Total of the 3 grades entered is 257
Class average is 85.67

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Implementing Sentinel-Controlled Iteration
 - To calculate a floating-point average in this example, we'll store the `total` as a `double`— C++'s default type for floating-point literals, such as `-7`, `0.0` or `98.6`.
 - Recall that integer division produces an integer result.
 - This program declares the variable `total` as a `double` (line 9). As you'll see, this will cause the average calculation to produce a floating-point result.
 - This program also stacks control statements in sequence—the `while` statement is followed in sequence by an `if...else` statement.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Program Logic for Sentinel-Controlled Iteration vs. Counter-Controlled Iteration
 - Line 9 initializes `gradeCounter` to 0 because no grades have been entered yet.
 - Remember that this program uses sentinel-controlled iteration to input the grades.
 - The program increments `gradeCounter` only when the user enters a valid grade.
 - Line 31 declares the `double` variable `average`, which stores the class average as a floating-point number.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Program Logic for Sentinel-Controlled Iteration vs. Counter-Controlled Iteration
 - In counter-controlled iteration, each loop iteration reads a value from the user for the specified number of iterations.
 - In sentinel-controlled iteration, the program prompts for and reads the first value before reaching the while.
 - ◆ This value determines whether the flow of control should enter the while's body.
 - ◆ If the condition is false, the user entered the sentinel value, so no grades were entered, and the body does not execute.
 - ◆ If the condition is true, the body executes, adding the grade value to the total and incrementing the `gradeCounter`.
 - ◆ Then lines 23–24 prompt for and input the next grade. At this point, program control reaches the loop's closing right brace (line 25).
 - ◆ Execution continues by testing the while's condition (line 18), using the most recently entered grade to determine whether the loop body should execute again.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Program Logic for Sentinel-Controlled Iteration vs. Counter-Controlled Iteration
 - The next `grade` is always input from the user immediately before the `while` condition is tested. This allows the program to determine whether the value just input is the sentinel value before processing it (i.e., adding it to the total).
 - ◆ If `grade` contains the sentinel value, the loop terminates without adding to the total. In a sentinel-controlled loop, prompts should remind the user of the sentinel.
 - After the loop terminates, the `if...else` statement at lines 30–42 executes.
 - ◆ The condition at line 30 determines whether any grades were input.
 - ◆ If none were input, the `if...else` statement's `else` part executes and displays the message "No grades were entered".
 - ◆ After the `if...else` executes, the program terminates.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Mixed-Type Expressions and Implicit Type Promotions
 - If at least one grade was entered, line 31
`double average{total / gradeCounter};`
divides `total` by `gradeCounter` to calculate the average.
 - Recall that `total` is a `double` and `gradeCounter` is an `int`.
 - For arithmetic, the compiler knows how to evaluate only expressions in which all the operand types are **identical**.
 - ◆ To ensure this, the compiler performs an operation called **promotion** (also called **implicit conversion**) on selected operands.
 - ◆ In an expression containing values of data types `double` and `int`, C++ promotes any `int` operands to `double` values.
 - ◆ So in line 31, C++ promotes a temporary copy of `gradeCounter`'s value to type `double`, then performs the division.

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Formatting Floating-Point Numbers

- Parameterized Stream Manipulator `setprecision`

- ◆ Line 36's call to `setprecision`—`setprecision(2)`—indicates that floating-point values should be output with two digits of precision to the right of the decimal point (e.g., 92.37).
 - ◆ `setprecision` is a parameterized stream manipulator because it requires an argument (in this case, 2) to perform its task.
 - ◆ Programs that use parameterized stream manipulators must include the header `<iomanip>` (line 3).

Formulating Algorithms: Sentinel-Controlled Iteration (Cont.)

- Formatting Floating-Point Numbers

- Nonparameterized Stream Manipulator fixed

- ◆ The non-parameterized stream manipulator fixed (line 36) does not require an argument and indicates that floating-point values should be output in **fixed-point format**.
 - ◆ This is as opposed to **scientific notation**, which displays a number from 1.0 up to, but not including, 10.0 multiplied by a power of 10.
 - So, in scientific notation, the value 3,100.0 is displayed as 3.1e+03
 - This format is useful for displaying very large or very small values (that is, values with negative exponents).
 - ◆ Fixed-point formatting forces a floating-point number to display without scientific notation.
 - Fixed-point formatting also forces the decimal point and trailing zeros to print, even if the value is a whole-number amount, such as 88.00.
 - Without the fixed-point formatting option, 88.00 prints as 88 without the trailing zeros and decimal point.
 - ◆ Stream manipulators `setprecision` and `fixed` perform **sticky settings**—all floating-point values formatted in your program will use those settings until you change them.

Formulating Algorithms: Sentinel-Controlled Iteration.

- Formatting Floating-Point Numbers

- Rounding Floating-Point Numbers

- ◆ When the stream manipulators `fixed` and `setprecision` are used, the formatted value is rounded to the number of decimal positions specified by the current precision.
 - The value in memory remains unaltered.
 - For the precision 2, 87.946 and 67.543 would be rounded to 87.95 and 67.54, respectively.
 - ◆ Lines 36 and 37 output the class average rounded to the nearest hundredth, with exactly two digits to the right of the decimal point.
 - The three grades entered during the program's execution `total 257`, which yields the `average 85.666...` and displays the rounded value 85.67.

Formulating Algorithms: Nested Control Statements

- We've seen that control statements can be stacked on top of one another (in sequence).
- In this case study, we examine nesting one control statement within another—the other structured way control statements can be connected.

Formulating Algorithms: Nested Control Statements (Cont.)

- Problem Statement

- Consider the following problem statement:

A college offers a course that prepares students for the state licensing exam for real estate brokers. Last year, 10 students who completed this course took the exam. The college wants to know how well its students did on the exam. You've been asked to write a program to summarize the results. You've been given a list of these 10 students. Next to each name is a 1 if the student passed the exam or a 2 if the student failed.

- Your program should analyze the results of the exam as follows:
 - ◆ Input each test result (i.e., a 1 or a 2). Display “Enter result” on the screen each time the program requests another test result.
 - ◆ Count the number of test results of each type.
 - ◆ Display a summary of the test results, indicating the number of students who passed and the number who failed.
 - ◆ If more than eight students pass, print “Bonus to instructor!”

Formulating Algorithms: Nested Control Statements (Cont.)

- Problem Statement Observations

- After reading the problem statement carefully, we make the following observations:
 - ◆ The program must process test results for 10 students.
 - We can counter-controlled iteration because we know the number of test results in advance.
 - ◆ Each time the program reads a test result, it must determine whether it's a 1 or a 2.
 - We test for a 1 in our algorithm. If the number is not a 1, we assume it's a 2.
 - ◆ Two counters are used to keep track of the exam results—
 - one to count the number of students who passed the exam and
 - one to count the number who failed.
 - ◆ After the program has processed all the results, it must decide whether more than eight students passed the exam.

Formulating Algorithms: Nested Control Statements (Cont.)

- Top-Down, Stepwise Refinement: Pseudocode Representation of the Top

- Let's proceed with top-down, stepwise refinement. We begin with a pseudocode representation of the top:

analyze exam results and decide whether a bonus should be paid

- Once again, the top is a complete representation of the program, but we'll likely need several refinements before the pseudocode can evolve naturally into a C++ program.

Formulating Algorithms: Nested Control Statements (Cont.)

- Top-Down, Stepwise Refinement: First Refinement

- Our first refinement is

initialize variables

input the 10 exam results, and count passes and failures

print a summary of the exam results and decide whether a bonus should be paid

- Here, too, though we have a complete representation of the entire program, further refinement is necessary.

- ◆ We now commit to specific variables. We need counters to record the passes and failures, a counter to control the looping process, and a variable to store the user input.
 - ◆ We do not need to initialize the variable that stores the user input because its value is read from the user during each loop iteration.

Formulating Algorithms: Nested Control Statements (Cont.)

- Top-Down, Stepwise Refinement: Second Refinement
 - The pseudocode statement *initialize variables* can be refined as follows:
 - initialize passes to zero*
 - initialize failures to zero*
 - initialize student counter to one*
 - Notice that only the counters are initialized at the start of the algorithm.

Formulating Algorithms: Nested Control Statements (Cont.)

- Top-Down, Stepwise Refinement: Second Refinement
 - The pseudocode statement *input the 10 exam results, and count passes and failures* requires a loop that successively inputs the result of each exam.
 - ◆ We know in advance that there are precisely 10 exam results, so counter-controlled looping is appropriate.
 - ◆ Inside the loop (i.e., nested within the loop), a double-selection structure will determine whether each exam result is a pass or a failure, then increment the appropriate counter.
 - The preceding pseudocode statement can be refined as follows:
while student counter is less than or equal to 10
prompt the user to enter the next exam result
input the next exam result
if the student passed
add one to passes
else
add one to failures
add one to student counter
 - We use blank lines to isolate the nested `if...else` statement, which improves readability.

Formulating Algorithms: Nested Control Statements (Cont.)

- Top-Down, Stepwise Refinement: Second Refinement

- The pseudocode statement

- print a summary of the exam results and decide whether a bonus should be paid*

- can be refined as follows:

- print the number of passes*

- print the number of failures*

- if more than eight students passed*

- print "Bonus to instructor!"*

Formulating Algorithms: Nested Control Statements (Cont.)

- Complete Second Refinement of the Pseudocode

```
initialize passes to zero
initialize failures to zero
initialize student counter to one

while student counter is less than or equal to 10
  prompt the user to enter the next exam result
  input the next exam result
  if the student passed
    add one to passes
  else
    add one to failures
  add one to student counter

print the number of passes
print the number of failures

if more than eight students passed
  print "Bonus to instructor!"
```

Formulating Algorithms: Nested Control Statements (Cont.)

- The next program implements the program with counter-controlled iteration and shows two sample executions.
 - Lines 7–9 and 15 declare the variables we use to process the examination results.
 - Lines 12–28 loop 10 times.
 - ◆ During each iteration, the loop inputs and processes one exam result. Notice that the `if...else` statement (lines 19–24) for processing each result is nested in the `while` statement.
 - ◆ If the result is 1, the `if...else` statement increments `passes`; otherwise, it assumes the result is 2 and increments `failures`.
 - ◆ Line 27 increments `studentCounter` before the loop condition is tested again at line 12.

Formulating Algorithms: Nested Control Statements (Cont.)

- Implementing the Program: Initialization

```
1 // Analysis of examination results using nested control statements.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // initializing variables in declarations
7     int passes{0};
8     int failures{0};
9     int studentCounter{1};
10 }
```

Formulating Algorithms: Nested Control Statements (Cont.)

- Implementing the Program: Processing

```
11 // process 10 students using counter-controlled loop
12 while (studentCounter <= 10) {
13     // prompt user for input and obtain value from user
14     cout << "Enter result (1 = pass, 2 = fail): ";
15     int result;
16     cin >> result;
17
18     // if...else is nested in the while statement
19     if (result == 1) {
20         passes = passes + 1;
21     }
22     else {
23         failures = failures + 1;
24     }
25
26     // increment studentCounter so loop eventually terminates
27     studentCounter = studentCounter + 1;
28 }
```

Formulating Algorithms: Nested Control Statements (Cont.)

- Implementing the Program: Termination

```
30 // termination phase; prepare and display results
31 cout << "Passed: " << passes << "\nFailed: " << failures << "\n";
32
33 // determine whether more than 8 students passed
34 if (passes > 8) {
35     cout << "Bonus to instructor!\n";
36 }
37 }
```

Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1

Passed: 9
Failed: 1
Bonus to instructor!

Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1

Passed: 6
Failed: 4

Formulating Algorithms: Nested Control Statements (Cont.)

- Implementing the Program
 - After 10 values have been input, the loop terminates, and line 31 displays the number of passes and failures.
 - The if statement in lines 34–36 determines whether more than eight students passed the exam and, if so, outputs the message "Bonus to instructor!"

Formulating Algorithms: Nested Control Statements (Cont.)

- Preventing Narrowing Conversions with Braced Initialization

- Consider the braced initialization

```
int studentCounter{1};  
which also can be written as  
int studentCounter = 1;
```

- For fundamental-type variables, braced initializers prevent **narrowing conversions** that could result in data loss.

- ◆ For example, the following declaration attempts to assign the double value 12.6 to the int variable x:

```
int x = 12.6;
```

- ◆ Here, C++ converts the double value to an int by **truncating** the floating-point part (.6), so this declaration assigns 12 to x.
- ◆ The narrowing conversion **loses data**. Compilers typically issue a warning for this but still compile the code.

Formulating Algorithms: Nested Control Statements.

- Preventing Narrowing Conversions with Braced Initialization

- However, using braced initialization, as in

```
int x{12.6};
```

yields a compilation error, helping you avoid a potentially subtle logic error.

- ◆ You'll still get a compilation error if you specify a whole-number double value, like 12.0.
- ◆ The initializer's type (double), not its value (12.0), determines whether a compilation error occurs.

- You might think that the following statement

```
int average{total / 10};
```

- ◆ contains a narrowing conversion, but total and 10 are both int values, so the initializer value is an int.
- ◆ If total were a double in the preceding statement or we used the double literal 10.0 for the denominator, then the initializer value would have type double, and the compiler would issue an error message for a narrowing conversion.

Compound Assignment Operators

- You can abbreviate the statement

`c = c + 3;`

with the addition compound assignment operator, `+=`, as

`c += 3;`

- The `+=` operator adds its right operand's value to the left-side variable's value, then stores the result in the left-side variable.
- Thus, the assignment expression `c += 3` adds 3 to `c`.

Compound Assignment Operators.

- The following table shows all the arithmetic compound assignment operators, sample expressions and explanations of what the operators do:

Operator	Sample expression	Explanation	Assigns
<i>Assume: int c = 3, d = 5, e = 4, f = 6, g = 12;</i>			
+=	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
-=	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
*=	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
/=	<code>f /= 3</code>	<code>f = f / 3</code>	2 to f
%=	<code>g %= 9</code>	<code>g = g % 9</code>	3 to g

Increment and Decrement Operators

- The following table summarizes C++'s two unary operators for adding 1 to or subtracting 1 from the value of a numeric variable—these are the unary increment operator, ++, and the unary decrement operator, --:

Operator	Operator name	Sample expression	Explanation
++	prefix increment	++number	Increment number by 1, then use the new value of number in the expression in which number resides.
++	postfix increment	number++	Use the current value of number in the expression in which number resides, then increment number by 1.
--	prefix decrement	--number	Decrement number by 1, then use the new value of number in the expression in which number resides.
--	postfix decrement	number--	Use the current value of number in the expression in which number resides, then decrement number by 1.

Increment and Decrement Operators (Cont.)

- An increment or decrement operator placed before (prefixed to) a variable is called the **prefix increment** or **prefix decrement** operator, respectively.
 - Using the prefix increment (or decrement) operator to add 1 to (or subtract 1 from) a variable preincrements (or predecrements) the variable.
 - The variable is incremented (or decremented) by 1, then its new value is used in the expression in which it appears.
- An increment or decrement operator placed after (postfixed to) a variable is called the **postfix increment** or **postfix decrement** operator, respectively.
 - Using the postfix increment (or decrement) operator to add 1 to (or subtract 1 from) a variable postincrements (or postdecrements) the variable.
 - The variable's value before the increment (or decrement) is used in the expression.
- Unlike binary operators, the unary increment and decrement operators by convention should be placed next to their operands, with no intervening spaces.

Increment and Decrement Operators (Cont.)

- Prefix Increment vs. Postfix Increment
 - The next program demonstrates the difference between the prefix increment and postfix increment versions of the ++ increment operator.
 - The decrement operator (--) works similarly.

Increment and Decrement Operators (Cont.)

• Prefix Increment vs. Postfix Increment

```
1 // Prefix increment and postfix increment operators.
2 #include <iostream>
3 using namespace std;
4
5 int main() {
6     // demonstrate postfix increment operator
7     int c{5};
8     cout << "c before postincrement: " << c << "\n"; // prints 5
9     cout << "    postincrementing c: " << c++ << "\n"; // prints 5
10    cout << " c after postincrement: " << c << "\n"; // prints 6
11
12    cout << "\n"; // skip a line
13
14    // demonstrate prefix increment operator
15    c = 5;
16    cout << " c before preincrement: " << c << "\n"; // prints 5
17    cout << "    preincrementing c: " << ++c << "\n"; // prints 6
18    cout << " c after preincrement: " << c << "\n"; // prints 6
19 }
```

c before postincrement: 5
postincrementing c: 5
c after postincrement: 6

c before preincrement: 5
preincrementing c: 6
c after preincrement: 6

Increment and Decrement Operators (Cont.)

- Prefix Increment vs. Postfix Increment

- Line 7 initializes the variable `c` to 5, and line 8 outputs `c`'s initial value.
- Line 9 outputs the value of the expression `c++`.
 - ◆ This expression postincrements the variable `c`, so `c`'s original value (5) is output, then `c`'s value is incremented (to 6). Thus, line 9 outputs `c`'s initial value (5) again.
 - ◆ Line 10 outputs `c`'s new value (6) to prove that the variable's value was incremented in line 9.
- Line 15 resets `c`'s value to 5, and line 16 outputs `c`'s value.
 - ◆ Line 17 outputs the value of the expression `++c`. This expression preincrements `c`, so its value is incremented. Then the new value (6) is output.
 - ◆ Line 18 outputs `c`'s value again to show that `c` is still 6 after line 18 executes.

Increment and Decrement Operators (Cont.)

- Simplifying Statements with the Arithmetic Compound Assignment, Increment and Decrement Operators

- The three assignment statements

```
passes = passes + 1;  
failures = failures + 1;  
student = student + 1;
```

- can be written more concisely with assignment operators as

```
passes += 1;  
failures += 1;  
student += 1;
```

- with preincrement operators as

```
++passes;  
++failures;  
++student;
```

- or with postincrement operators as

```
passes++;  
failures++;  
student++;
```

Increment and Decrement Operators (Cont.)

- Simplifying Statements with the Arithmetic Compound Assignment, Increment and Decrement Operators
 - When incrementing or decrementing a variable in a statement by itself, the prefix increment and postfix increment forms have the same effect, and the prefix decrement and postfix decrement forms have the same effect.
 - ◆ Only when a variable appears in the context of a larger expression does preincrementing or postincrementing the variable have a different effect (and similarly for predecrementing or postdecrementing).
 - Attempting to use the increment or decrement operator on an expression other than one to which a value can be assigned is a compilation error.
 - ◆ For example, writing `++(x + 1)` is a syntax error, because `(x + 1)` is not a variable.

Increment and Decrement Operators.

- Operator Precedence and Grouping

- The following table shows the precedence and grouping of the operators introduced to this point.

Operators	Grouping
++ (<i>postfix</i>) -- (<i>postfix</i>)	left to right
+ - ++ (<i>prefix</i>) -- (<i>prefix</i>)	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
?:	right to left
= += -= *= /= %=	right to left

Fundamental Types Are Not Portable

- You can view the complete list of C++ fundamental types and their typical ranges at:
<https://en.cppreference.com/w/cpp/language/types>
- In C and C++, an `int` on one machine might be represented by 16 bits (2 bytes) of memory, on a second machine by 32 bits (4 bytes), and on another machine by 64 bits (8 bytes).
 - For this reason, code using integers is not always portable across platforms.
 - You could write multiple versions of your programs to use different integer types on different platforms.
 - Or you could use techniques to achieve various levels of portability.
- Among C++'s integer types are `int`, `long` and `long long`.
 - The C++ standard requires type `int` to be at least 16 bits, type `long` to be at least 32 bits and type `long long` to be at least 64 bits.
 - The standard also requires that an `int`'s size be less than or equal to a `long`'s size and that a `long`'s size be less than or equal to a `long long`'s size.
 - Such “squishy” requirements create portability challenges but allow compiler implementers to optimize performance by matching fundamental types sizes to your machine's hardware.

Objects Natural Case Study: Super-Sized Integers

- The explosive growth of Internet communications and data storage on Internet-connected devices has dramatically increased the importance of privacy and security.
- Cryptography has been used for thousands of years to encode data, making it difficult (and hopefully impossible) for unauthorized users to read, which is critically important in today's connected world.
- Most websites (including deitel.com) now use HTTPS (HyperText Transfer Protocol Secure) to encrypt and decrypt your Internet interactions.

Objects Natural Case Study: Super-Sized Integers (Cont.)

- Some Applications Need Integers Outside a long long Integer's Range
 - Many cryptography algorithms perform calculations with integers far larger than we can store using even C++'s long long type. On most systems, a long long integer can store values in the range -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 —a maximum of 19 decimal digits (which can represent integers in the quintillions).
 - One of the most popular encryption/decryption schemes is [RSA Public-Key Cryptography](#), commonly used to secure data transmitted over the Internet. Industrial-strength RSA implementations work with [enormous prime numbers consisting of hundreds of digits](#).
 - A key reason why RSA is so secure is the sheer amount of time required to factor the product of those primes, even for today's most powerful supercomputers. Public-key cryptography also is used to secure the blockchain technology behind cryptocurrencies like Bitcoin.

Objects Natural Case Study: Super-Sized Integers (Cont.)

- Integers Outside the long long Range Require Custom Programming
 - So how can we manipulate huge integers with arbitrary numbers of digits? The C++ standard library does not provide such capabilities, so applications requiring integers outside long long's range require custom programming.
 - We could use the C++' class mechanism to develop a custom class that handles huge-integer arithmetic—but that could take months of effort.
 - The Objects Natural approach encourages us to check first for free, open-source class libraries to see if such a “huge-integer” class already exists.

Objects Natural Case Study: Super-Sized Integers (Cont.)

- Huge Integers with the Boost Multiprecision Open Source Library's `cpp_int` Class
 - Indeed, there are many preexisting C++ open-source classes for creating and manipulating huge integers.
 - For this example, we'll demonstrate the Boost Multiprecision open source library's `cpp_int` class from:
<https://github.com/boostorg/multiprecision/>

Objects Natural Case Study: Super-Sized Integers (Cont.)

- Huge Integers with the Boost Multiprecision Open Source Library's `cpp_int` Class
 - With the Objects Natural approach, we'll conveniently use class `cpp_int` without having to understand how it's implemented.
 - We include its header (line3), easily create `cpp_int` objects (lines 10–11), then conveniently manipulate them in our code with familiar arithmetic operations (lines 21–28).

Objects Natural Case Study: Super-Sized Integers (Cont.)

[illegible]

Objects Natural Case Study: Super-Sized Integers (Cont.)

- `using` Declaration

- We have showed that a `using` declaration can eliminate the need to repeat the `"std::"` prefix for C++ standard library components, such as `std::cin` and `std::cout`.
- The Boost Multiprecision library's capabilities must be similarly qualified with `boost::multiprecision::` as in `boost::multiprecision::cpp_int`
- Line 7's `using` declaration enables you to use the class name `cpp_int` rather than the fully qualified class name `boost::multiprecision::cpp_int`.

Objects Natural Case Study: Super-Sized Integers (Cont.)

- Initializing `cpp_ints`

- Lines 10 and 11 initialize the `cpp_ints` `value1` and `value2` with a string and a long long, respectively.
 - ◆ For cases in which the initializer value for a `cpp_int` exceeds the maximum integer values C++ supports, such as the 30-digit integer in line 10, you initialize a `cpp_int` with a string representing an integer value.
 - ◆ You may also use any valid integer as an initializer, as in line 11, where we chose to use the maximum value of a long long. The LL (“el el”) suffix at the end of the value indicates that it’s a long long integer.
 - ◆ Line 12 also defines the fundamental type `int` variable `value3`, which we’ll use momentarily.
 - ◆ Lines 14–17 display these variables’ initial values.

Objects Natural Case Study: Super-Sized Integers (Cont.)

- Performing Arithmetic as Simply as Arithmetic with Fundamental Integer Types
 - Class `cpp_int` supports the same arithmetic operations as C++'s `int`, `long` and `long long` fundamental types
 - Lines 20–23 demonstrate addition, subtraction and multiplication with `cpp_int`s `value1` and `value2`.
 - ◆ `cpp_int` correctly handles integers of any number of digits. Each result is a value with more digits than can be represented by C++'s fundamental integer types.
 - ◆ The multiplication in line 23, for example, produces a 48-digit integer.
 - Though a `cpp_int` can represent any integer value, it is not built into your system's hardware (i.e., it's not a fundamental type). So calculations with `cpp_int` run (possibly much) **slower** than those with fundamental integer types.

Objects Natural Case Study: Super-Sized Integers (Cont.)

- How Do C++'s Arithmetic Operators and the `<<` Operator Understand `cpp_int`?
 - The compiler already knows how to use arithmetic operators and the `<<` operator with fundamental numeric types, but it has to be taught how to handle those operators for custom `cpp_int` class objects.
 - Later we'll discuss in detail how to do this with a powerful technique called [operator overloading](#).
- Performing Mixed-Type Arithmetic Between a `cpp_int` and an Integer
 - Class `cpp_int` also supports mixed-type expressions containing `cpp_int`s and integers of C++'s fundamental types. Lines 26–28 demonstrate this by multiplying the `cpp_int` object `value1` by the `int` variable `value3`, and by the `int` value 17.

Objects Natural Case Study: Super-Sized Integers.

- Compiling and Running the Example
 - Install boost
 - ◆ Download boost
 - ◆ Run bootstrap.bat mingw b2.exe (optional)
 - ◆ Run b2.exe --prefix=build (optional)
 - Add the path of your boost library to the search directory